UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

Interrupções

Docente: Prof. Dr. Ruhan Pontes Policarpo de Souza
Discente: Carlos Gabriel Baratieri — 2706598
Discente: Luiz Fernando Coimbra Hintz — 2559145
Turma: C51B

O presente documento tem como objetivo relatar a quarta prática laboratorial da disciplina de Sistemas Microcontrolados, envolvendo interrupções.

Sistemas Microcontrolados — EC45C
29/09/2026

2 de outubro de 2026 — 17:37
Discente: Carlos Gabriel Baratieri
Discente: Luiz F. Coimbra Hintz

1 Introdução

O objetivo deste experimento foi implementar o funcionamento de interrupções no microcontrolador PIC18F4550, utilizando o display LCD alfanumérico 16x2. Dessa forma, foi desenvolvido um firmware usando o MPLAB X IDE 5.20, testado e simulado no ambiente Proteus e implementado no kit didático XM118 – Exsto.

O firmware desenvolvido utiliza interrupções de baixa e alta prioridade, associadas às entradas `INT1` e `INT0`. Durante a execução, o display LCD alfanumérico 16x2 apresenta a data na primeira linha e um contador de segundos na segunda linha. Quando uma interrupção é acionada, o funcionamento é interrompido para apresentar uma mensagem correspondente à prioridade da interrupção durante um período determinado.

2 Materiais e Métodos

Foram utilizados o microcontrolador PIC18F4550, o kit didático XM118 – Exsto, contendo o PIC18F4550, teclado/botões de acionamento e display LCD alfanumérico 16x2, um computador com o MPLAB X IDE 5.20, o software Proteus, cabos para conexão e um osciloscópio.

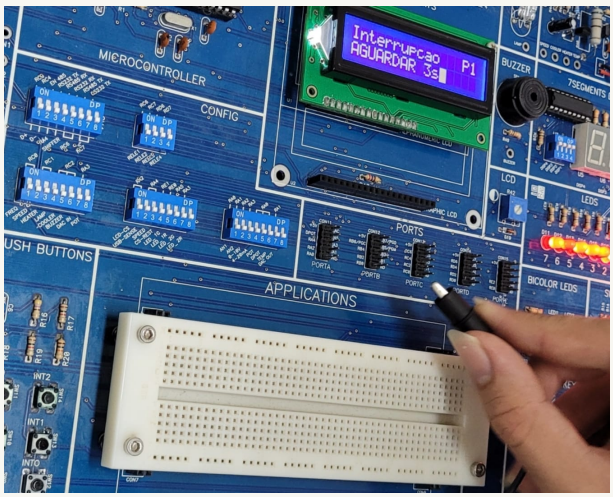


Figura 1: Kit didático XM118 – Exsto

Foi utilizado o kit didático XM118 – Exsto, apresentado na Figura 1, para a realização dos testes práticos do experimento. Durante os testes, também foi utilizado um osciloscópio, apresentado na Figura 2, para auxiliar na análise dos sinais e na verificação do funcionamento dos botões. Durante os testes, o botão `RB0` estava em curto, logo, aterramos ele para seu funcionamento.

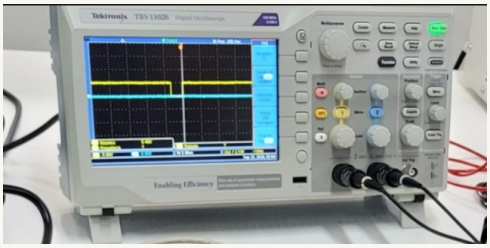


Figura 2: Osciloscópio

O firmware foi desenvolvido utilizando os recursos de interrupção do PIC18F4550 e o display LCD alfanumérico 16x2. Durante a execução normal do programa, o display apresenta a data na primeira linha e um contador de segundos na segunda linha. Também foram configuradas interrupções de baixa e alta prioridade, utilizando as entradas `INT1` e `INT0`. Quando uma interrupção de baixa prioridade é acionada, o display apresenta a mensagem

correspondente durante 5 segundos. Já para a interrupção de alta prioridade, é apresentada a mensagem correspondente durante 3 segundos.

Após a implementação e compilação do código, foram realizados testes e simulações no Proteus para verificar o funcionamento do firmware e das interrupções. Posteriormente, o código foi gravado no PIC18F4550 do kit didático XM118 – Exsto para a realização dos testes práticos. Foi desenvolvido o esquemático no Proteus, como pode ser observado na Figura 3.

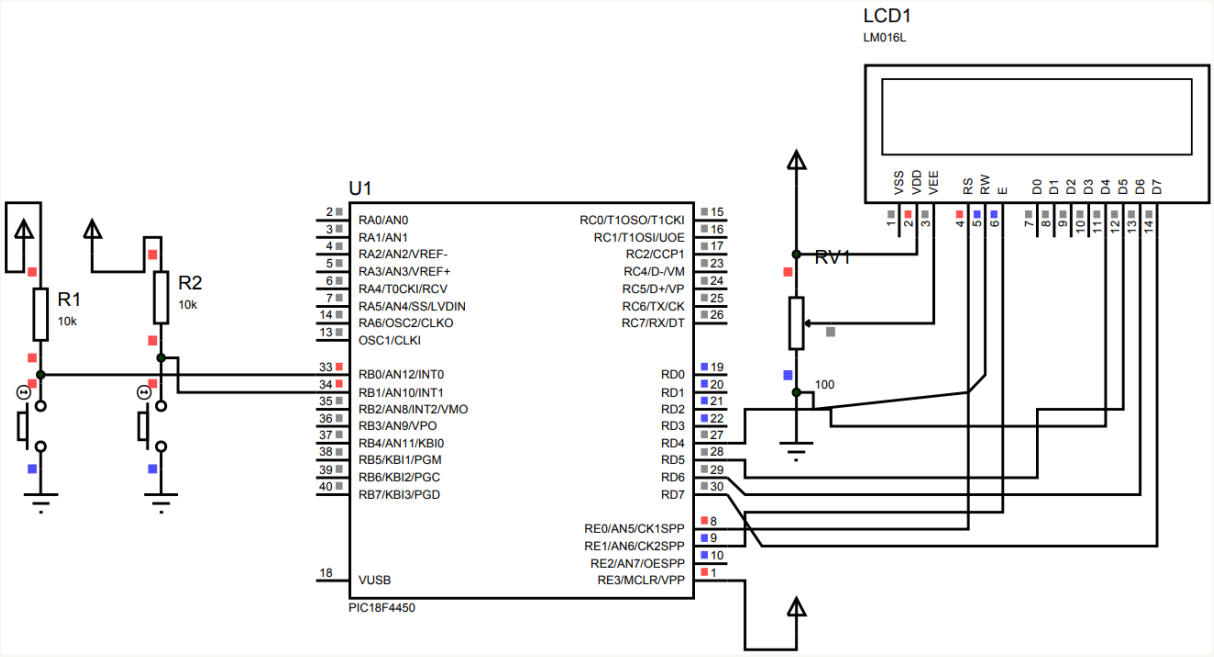


Figura 3: Esquemático

Abaixo, é apresentado o compilador utilizado no desenvolvimento do firmware, conforme a Figura 4.

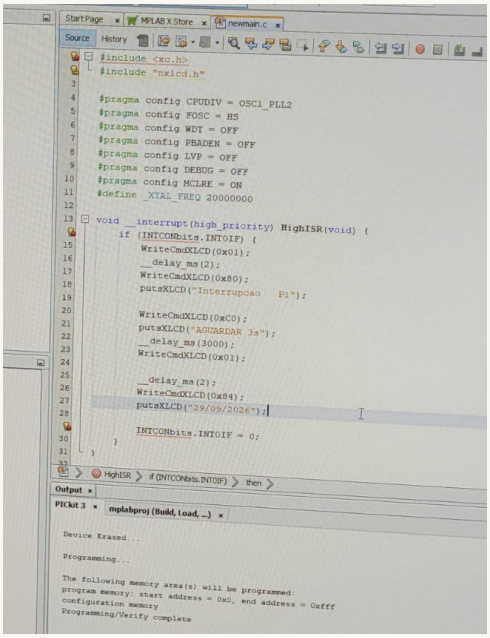


Figura 4: MPLAB X IDE 5.20



3 Resultados

Abaixo está o código que foi testado tanto no kit, quanto no Proteus.

```c
#include <xc.h>
#include "nxlcd.h"

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON

#define _XTAL_FREQ 20000000

void __interrupt(high_priority) HighISR(void) {
    if (INTCONbits.INT0IF) {
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x80);
        putsXLCD("Interrupcao   P1");
        WriteCmdXLCD(0xC0);
        putsXLCD("AGUARDAR 3s");
        __delay_ms(3000);
        if (!INTCON3bits.INT1IF) {
            WriteCmdXLCD(0x01);
            __delay_ms(2);
            WriteCmdXLCD(0x83);
            putsXLCD("29/09/2026");
        } else {
            WriteCmdXLCD(0x8F);
            putsXLCD("0");
            WriteCmdXLCD(0xC9);
            putsXLCD("5");
        }
        INTCONbits.INT0IF = 0;
    }
}

void __interrupt(low_priority) LowISR(void) {
    if (INTCON3bits.INT1IF) {
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x80);
        putsXLCD("Interrupcao   P0");
        WriteCmdXLCD(0xC0);
        putsXLCD("AGUARDAR 5s");
        __delay_ms(5000);
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x83);
        putsXLCD("29/09/2026");
        INTCON3bits.INT1IF = 0;
    }
}

void main(void) {
    unsigned char segundos = 0;

    TRISD = 0;
    TRISE = 0;

    TRISBbits.TRISB0 = 1;
    TRISBbits.TRISB1 = 1;

    RCONbits.IPEN = 1;
    INTCONbits.GIEH = 1;
    INTCONbits.GIEL = 1;
    INTCONbits.INT0IE = 1;
    INTCONbits.INT0IF = 0;
    INTCON3bits.INT1IE = 1;
    INTCON3bits.INT1IF = 0;
    INTCON3bits.INT1IP = 0;

    OpenXLCD(FOUR_BIT & LINES_5X7);
    WriteCmdXLCD(0x01);
    __delay_ms(2);

    WriteCmdXLCD(0x83);
    putsXLCD("29/09/2026");

    while (1) {
        WriteCmdXLCD(0xC2);
        putsXLCD("SEGUNDOS: ");
        putcXLCD((segundos / 10) + '0');
        putcXLCD((segundos % 10) + '0');
        segundos++;
        __delay_ms(1000);
        if (segundos >= 60) {
            segundos = 0;
        }
    }
}
```



Primeiro são definidas as bibliotecas e as configurações iniciais do microcontrolador. Usamos novamente a `nxlcd.h` para manipulação de alguns comandos do display. Os pragmas são configurações dadas pelo professor e o macro da frequência também.

```c
#include <xc.h>
#include "nxlcd.h"

#pragma config CPUDIV = OSC1_PLL2
#pragma config FOSC = HS
#pragma config WDT = OFF
#pragma config PBADEN = OFF
#pragma config LVP = OFF
#pragma config DEBUG = OFF
#pragma config MCLRE = ON

#define _XTAL_FREQ 20000000
```

Toda a parte lógica vai ser explicada de uma forma diferente, sendo baseada no enunciado dado pelo professor.

> Enquanto o microcontrolador estiver processando o `loop` infinito [...]

Essa parte se refere ao trecho abaixo, onde o microcontrolador precisa ficar verificando as ações constantemente.

```c
while (1) {
    // [...]
}
```

> Enquanto o microcontrolador estiver processando o `loop` infinito, o display LCD alfanumérico deve apresentar a data na primeira linha [...]

Abaixo, o comando `WriteCmdXLCD()` é utilizado para posicionar o cursor para escrevermos na tela. Nesse caso, ele utiliza como referência a tabela de posições do LCD abaixo.

80	**81**	**82**	**83**	**84**	**85**	**86**	**87**	**88**	**89**	**8A**	**8B**	**8C**	**8D**	**8E**	**8F**
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF

O comando `OpenXLCD(FOUR_BIT & LINES_5X7)` configura o LCD para utilizar o modo de 4 bits de dados e caracteres no formato 5x7. Já o comando `WriteCmdXLCD(0x01)` limpa o conteúdo do display, e o `__delay_ms(2)` realiza um atraso de 2 milissegundos para aguardar a execução do comando.

No código, o comando `WriteCmdXLCD(0x83)` posiciona o cursor na posição `0x83`, que corresponde à posição utilizada para centralizar a data na primeira linha. Em seguida, o comando `putsXLCD()` escreve a data no display.

```c
OpenXLCD(FOUR_BIT & LINES_5X7);
WriteCmdXLCD(0x01);
__delay_ms(2);

WriteCmdXLCD(0x83);
putsXLCD("29/09/2026");

while (1) {
    // [...]
}
```

> Enquanto o microcontrolador estiver processando o `loop` infinito, o display LCD alfanumérico deve apresentar a data na primeira linha, e um contador de segundos na segunda linha, com a seguinte mensagem: `"Segundos: XX"` [...]

Basicamente, o comando `WriteCmdXLCD(0xC2)` vai selecionar a posição inicial para a escrita da mensagem na segunda linha, enquanto o comando `putsXLCD()` escreve o texto `"SEGUNDOS: "` no display.

```
OpenXLCD(FOUR_BIT & LINES_5X7);
WriteCmdXLCD(0x01);
__delay_ms(2);

WriteCmdXLCD(0x83);
putsXLCD("29/09/2026");

while (1) {
    WriteCmdXLCD(0xC2);
    putsXLCD("SEGUNDOS: ");
    // [...]
}
```

> Enquanto o microcontrolador estiver processando o `loop` infinito, o display LCD alfanumérico deve apresentar a data na primeira linha, e um contador de segundos na segunda linha, com a seguinte mensagem: `"Segundos: XX"`, sendo XX um valor de 0 a 59. O contador de segundos deve ser incrementado a cada 1000 milissegundos. O texto em ambas as linhas deve estar centralizado.

Os comandos `putcXLCD()` vão basicamente colocar os dois caracteres referentes ao valor de segundos após o texto apresentado anteriormente. Eles recebem como argumento um cálculo matemático que separa o valor das dezenas e das unidades. No primeiro comando, `segundos / 10` obtém o valor das dezenas, enquanto no segundo, `segundos \% 10` obtém o valor das unidades. O `+ '0'` é utilizado para converter o valor numérico no caractere correspondente.

Depois disso, o comando `segundos++` incrementa o contador, e o `__delay_ms(1000)` faz o microcontrolador aguardar 1000 milissegundos antes da próxima atualização.

```
OpenXLCD(FOUR_BIT & LINES_5X7);
WriteCmdXLCD(0x01);
__delay_ms(2);

WriteCmdXLCD(0x83);
putsXLCD("29/09/2026");

while (1) {
    WriteCmdXLCD(0xC2);
    putsXLCD("SEGUNDOS: ");
    putcXLCD((segundos / 10) + '0');
    putcXLCD((segundos % 10) + '0');
    segundos++;
    __delay_ms(1000);
    // [...]
}
```

O `if` delimita o valor máximo do contador, verificando se ele atingiu ou ultrapassou 60. Quando isso acontece, o valor de `segundos` é zerado, fazendo com que o contador volte para 0 e permaneça no intervalo de 0 a 59.

```
OpenXLCD(FOUR_BIT & LINES_5X7);
WriteCmdXLCD(0x01);
__delay_ms(2);

WriteCmdXLCD(0x83);
putsXLCD("29/09/2026");

while (1) {
    WriteCmdXLCD(0xC2);
    putsXLCD("SEGUNDOS: ");
    putcXLCD((segundos / 10) + '0');
    putcXLCD((segundos % 10) + '0');
    segundos++;
    __delay_ms(1000);

    if (segundos >= 60) {
        segundos = 0;
    }
}
```

Agora, vamos falar sobre a parte das interrupções. Todos os códigos são baseados nos exemplos que o professor demonstrou em sala de aula, principalmente nas configurações das interrupções.



> Quando ocorrer uma interrupção de baixa prioridade, o display LCD deve apresentar na primeira linha `"Interrupção - P0"` [...]

Essa parte se refere à função `LowISR()`, que é a função responsável pelo tratamento da interrupção de baixa prioridade. O `if` verifica se a interrupção `INT1` ocorreu por meio da flag `INT1IF`.

```
void __interrupt(low_priority) LowISR(void) {
    if (INTCON3bits.INT1IF) {
        // [...]
    }
}
```

> Quando ocorrer uma interrupção de baixa prioridade, o display LCD deve apresentar na primeira linha `"Interrupção - P0"`, e na segunda linha `"Aguardar 5 segundos"` [...]

Dentro do `if`, primeiro o comando `WriteCmdXLCD(0x01)` limpa o conteúdo do display. Em seguida, o cursor é posicionado na primeira linha com `WriteCmdXLCD(0x80)`, e a mensagem `"Interrupcao P0"` é apresentada com `putsXLCD()`. Depois, o cursor é posicionado na segunda linha com `WriteCmdXLCD(0xC0)`, onde é apresentada a mensagem `"AGUARDAR 5s"`.

```
void __interrupt(low_priority) LowISR(void) {
    if (INTCON3bits.INT1IF) {
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x80);
        putsXLCD("Interrupcao   P0");
        WriteCmdXLCD(0xC0);
        putsXLCD("AGUARDAR 5s");
    }
}
```

> Quando ocorrer uma interrupção de baixa prioridade, o display LCD deve apresentar na primeira linha `"Interrupção - P0"`, e na segunda linha `"Aguardar 5 segundos"`. A exibição deve ter duração de 5 segundos, e sair da rotina de tratamento de interrupção após este período.

Para manter a mensagem no display durante os 5 segundos, usamos `__delay_ms(5000)`. Após esse período, o display é limpo e a data é apresentada novamente.

```
void __interrupt(low_priority) LowISR(void) {
    if (INTCON3bits.INT1IF) {
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x80);
        putsXLCD("Interrupcao   P0");
        WriteCmdXLCD(0xC0);
        putsXLCD("AGUARDAR 5s");
        __delay_ms(5000);
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x83);
        putsXLCD("29/09/2026");
    }
}
```

Por fim, a flag `INT1IF` é zerada para indicar que a interrupção `INT1` já foi atendida.

```
void __interrupt(low_priority) LowISR(void) {
    if (INTCON3bits.INT1IF) {
        // [...]
        INTCON3bits.INT1IF = 0;
    }
}
```

> A interrupção de baixa prioridade será a interrupção `INT1` e a interrupção de alta prioridade será a interrupção `INT0`.



> Quando ocorrer uma interrupção de alta prioridade, o display LCD deve apresentar o valor `"Interrupção - P1"` [...]

Nesse caso, a função `HighISR()` é responsável por tratar a interrupção de alta prioridade. O comando `if` verifica se a flag da interrupção `INT0` está ativa.

```
void __interrupt(high_priority) HighISR(void) {
    if (INTCONbits.INT0IF) {
    }
}
```

> Quando ocorrer uma interrupção de alta prioridade, o display LCD deve apresentar o valor `"Interrupção - P1"` na primeira linha, e `"Aguardar 3 segundos"` na segunda linha [...]

Para apresentar as mensagens no display, o comando `WriteCmdXLCD(0x01)` limpa o LCD. Em seguida, `WriteCmdXLCD(0x80)` posiciona o cursor na primeira linha e `putsXLCD()` escreve a mensagem. Depois, `WriteCmdXLCD(0xC0)` posiciona o cursor na segunda linha, onde a segunda mensagem é apresentada.

```
void __interrupt(high_priority) HighISR(void) {
    if (INTCONbits.INT0IF) {
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x80);
        putsXLCD("Interrupcao   P1");
        WriteCmdXLCD(0xC0);
        putsXLCD("AGUARDAR 3s");
    }
}
```

> Quando ocorrer uma interrupção de alta prioridade, o display LCD deve apresentar o valor `"Interrupção - P1"` na primeira linha, e `"Aguardar 3 segundos"` na segunda linha. A exibição desta mensagem deve ter duração total de 3 segundos, e sair da rotina de tratamento de interrupção após este período.

Para manter a mensagem no display durante os 3 segundos, utiliza-se `__delay_ms(3000)`. Após esse período, o display é limpo e a data é apresentada novamente.

Caso a flag `INT1IF` esteja ativa, significa que existe uma interrupção de baixa prioridade pendente. Nesse caso, o comando `else` evita que a tela seja limpa e apresenta diretamente os valores correspondentes à interrupção anterior.

```
void __interrupt(high_priority) HighISR(void) {
    if (INTCONbits.INT0IF) {
        WriteCmdXLCD(0x01);
        __delay_ms(2);
        WriteCmdXLCD(0x80);
        putsXLCD("Interrupcao   P1");
        WriteCmdXLCD(0xC0);
        putsXLCD("AGUARDAR 3s");
        __delay_ms(3000);
        if (!INTCON3bits.INT1IF) {
            WriteCmdXLCD(0x01);
            __delay_ms(2);
            WriteCmdXLCD(0x83);
            putsXLCD("29/09/2026");
        } else {
            WriteCmdXLCD(0x8F);
            putsXLCD("0");
            WriteCmdXLCD(0xC9);
            putsXLCD("5");
        }
        INTCONbits.INT0IF = 0;
    }
}
```

O comando `INTCONbits.INT0IF = 0` limpa a flag da interrupção de alta prioridade, permitindo que uma nova interrupção `INT0` seja detectada. Tanto o `if`, quanto o `else` podem ser consideradas uma gambiarra técnica.



Abaixo temos o código restante. A variável `unsigned char segundos` é declarada para realizar os cálculos já explicados anteriormente. As variáveis `TRISD` e `TRISE` são configuradas para definir os pinos dos `PORTD` e `PORTE` como saídas. Já `TRISBbits.TRISB0` e `TRISBbits.TRISB1` configuram os pinos `RB0` e `RB1` como entradas, que serão utilizados pelas interrupções.

> `RB0` — `INT0` (alta prioridade)
> `RB1` — `INT1` (baixa prioridade)

```
void main(void) {
    unsigned char segundos = 0;

    TRISD = 0;
    TRISE = 0;

    TRISBbits.TRISB0 = 1;
    TRISBbits.TRISB1 = 1;

    RCONbits.IPEN = 1;
    INTCONbits.GIEH = 1;
    INTCONbits.GIEL = 1;
    INTCONbits.INT0IE = 1;
    INTCONbits.INT0IF = 0;
    INTCON3bits.INT1IE = 1;
    INTCON3bits.INT1IF = 0;
    INTCON3bits.INT1IP = 0;

    // [...]
}
```

O `RCONbits.IPEN` habilita as interrupções com nível de prioridade, enquanto `INTCONbits.GIEH` e `INTCONbits.GIEL` habilitam as interrupções de alta e baixa prioridade. Com essa configuração, o vetor de interrupção de alta prioridade está localizado no endereço `0x08`, enquanto o vetor de baixa prioridade está localizado no endereço `0x18`. Os `INTCONbits.INT0IE` e `INTCON3bits.INT1IE` habilitam as interrupções `INT0` e `INT1`, enquanto `INTCONbits.INT0IF = 0` e `INTCON3bits.INT1IF = 0` limpam suas respectivas flags. Por fim, o `INTCON3bits.INT1IP` define a `INT1` como uma interrupção de baixa prioridade. As flags servem para indicar que o evento da interrupção ocorreu.

4 Conclusão

Dessa forma, foi abordada a implementação de um firmware sobre o funcionamento das interrupções de alta e baixa prioridade. Foi realizada a simulação por meio do Proteus e um teste prático no kit didático XM118 – Exsto.

O firmware apresentou bom funcionamento, realizando a contagem dos segundos e o acionamento das interrupções `INT0` e `INT1`, respeitando suas respectivas prioridades e tempos. Devido ao problema de curto no botão `RB0`, foi necessário realizar um aterramento para seu funcionamento. Mas, os objetivos propostos para o experimento foram alcançados.